Data Collection and Imitation Learning

Course home · 简体中文 · PDF

Item	Details
Applicability	Demonstrations, BC, DAgger, ACT and generative chunk policies
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

Verification scope: Methods survey and proposed experiments; no validated training results. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- 1. From Demonstrations to a Trainable Dataset
  - 1.1 What One Demonstration Must Contain
  - 1.2 Time and Frames Come Before Architecture
- 2. Collection and Quality Control
  - 2.1 Beyond Successful Demonstrations
  - 2.2 Splits Matter More Than a Headline Score
- 3. Imitation Learning: BC to Interactive Correction
  - 3.1 Why Low Training Loss May Still Fail
  - 3.2 What DAgger Adds
- 4. From Single Actions to Generative Action Chunks
  - 4.1 ACT in One Picture
  - 4.2 Diffusion Policy and Flow Matching
- 5. Augmentation, Recalibration and Simulation
  - 5.1 Contact Events and Recovery Samples
  - 5.2 Safe Augmentation Boundaries
- 6. A Shared Panda Tabletop Experiment
  - 6.1 Minimum Experiment Matrix
  - 6.2 Debugging Checklist
- 7. Unified Data Model: From Demonstrations to Action Chunks
  - 7.1 Distinguish Latent State, Observations, and Commands
  - 7.2 The Action-Chunk Prediction Problem
  - 7.3 Time Alignment, Frames, and Action Normalization
- 8. Behavior Cloning: Objectives, Intuition, and Failure Modes
  - 8.1 Deterministic and Probabilistic BC

    - 8.2 Covariate Shift and Compounding Error
    - 8.3 Chunk Loss and Smoothness Trade-offs
- 9. Interactive Imitation and Recovering from Failure
    - 9.1 DAgger: Expert Queries on Learner-Visited States
    - 9.2 Failures, Recoveries, and Event-Biased Sampling
- 10. ACT, Diffusion Policy, Flow Matching, and RTC: Training vs. Execution
    - 10.1 ACT: Conditional VAE and Transformer Chunks
    - 10.2 Diffusion Policy: Predict Noise in Action Space
    - 10.3 Flow Matching: Generative Velocity Is Not Joint Velocity
    - 10.4 Fair Comparisons Against BC
    - 10.5 Why RTC Matters: From Synchronous Chunks to Asynchronous Control
    - 10.6 Inference-Time RTC: Inpainting With a Frozen Prefix and Soft Overlap
    - 10.7 Training-Time RTC: Learn the Prefix Constraint From Demonstrations
    - 10.8 Comparing RTC Against Temporal Ensembling and Naive Async
- 11. Human Video Retargeting, Contact, and Augmentation
    - 11.1 Human-to-Robot Is Not Pixel Imitation
    - 11.2 Augmentation Must Preserve Geometry and Physics
- 12. Training and Evaluation Protocol
    - 12.1 Offline Metrics, Closed-Loop Success, and Recovery
    - 12.2 A Shared Panda Tabletop Benchmark
- 13. Where This Fits and References

Learning outcomes: Identify demonstration quality risks; understand BC, DAgger, ACT, and diffusion/flow chunk training; distinguish inference-time and training-time RTC; evaluate timing, continuity, and task success.

1. From Demonstrations to a Trainable Dataset

Imitation learning is not just storing a successful video. It turns observations, robot states, actions and task conditions into aligned and reproducible supervision. A closed-loop robot ultimately learns a conditional policy $\pi_\theta(a_t \mid o_{\le t}, g)$, where g is the goal.

1.1 What One Demonstration Must Contain

Each episode should at least log task/scene IDs, image timestamps, RGB or RGB-D, joint feedback, executed actions, action reference frame, control mode, success, and exceptional events. **The action label must identify the command actually issued**; do not mix commanded joint targets, measured positions, and joint velocities. For vision, specify whether camera extrinsics are fixed or time-varying.

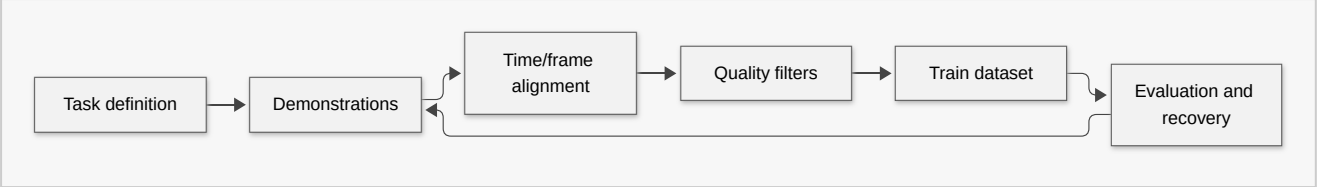
1.2 Time and Frames Come Before Architecture

Use one clock basis or verified timestamp alignment. Define T_B^C, T_B^E and T_B^O under one transform convention. Cameras often run slower than proprioception; match past observations to action

timestamps or use documented delay-aware interpolation, never future frames.

2. Collection and Quality Control

Common sources include teleoperation, kinesthetic teaching, scripted experts, simulated RL experts, and retargeted human video. They differ in throughput, action quality, contact fidelity, and hardware cost.



Practical principle: start with a smaller coherent dataset rather than accumulating large volumes of unaligned video.

2.1 Beyond Successful Demonstrations

Retain informative failures, corrective behavior, and recovery segments, with event boundaries such as first contact, slip, and regrasp. Do not blindly imitate every failed action: distinguish useful recovery demonstrations from unsafe commands. Record consent, data permissions, licensing, and robot safety review.

2.2 Splits Matter More Than a Headline Score

Split by object instance, scene layout, operator, initial configuration, and collection session. **Never put chunks from the same continuous trajectory in both train and test sets.** Report episode success, not only per-action MSE. Group synchronized views and adjacent frames together to prevent leakage.

3. Imitation Learning: BC to Interactive Correction

Behavior cloning (BC) fits actions from demonstrations. Deterministic regression can minimize $\mathcal{L}_{BC} = \mathbb{E}_{(o,a)\sim D}\|\pi_\theta(o) - a\|_2^2$; stochastic policies maximize conditional action likelihood. Neither objective automatically fixes deployment-time distribution shift.

3.1 Why Low Training Loss May Still Fail

In closed-loop execution, small action errors change future observations and push the robot outside the demonstration distribution. Record state drift, failed contacts, interventions, and recovery from perturbed states. Safe, supervised recovery demonstrations can expand coverage.

3.2 What DAgger Adds

DAgger collects states visited by the learner, queries an expert for suitable actions, and aggregates labeled data. A real robot needs safe intervention procedures. Offline retraining without expert relabeling of learner-visited states should not be called DAgger.

4. From Single Actions to Generative Action Chunks

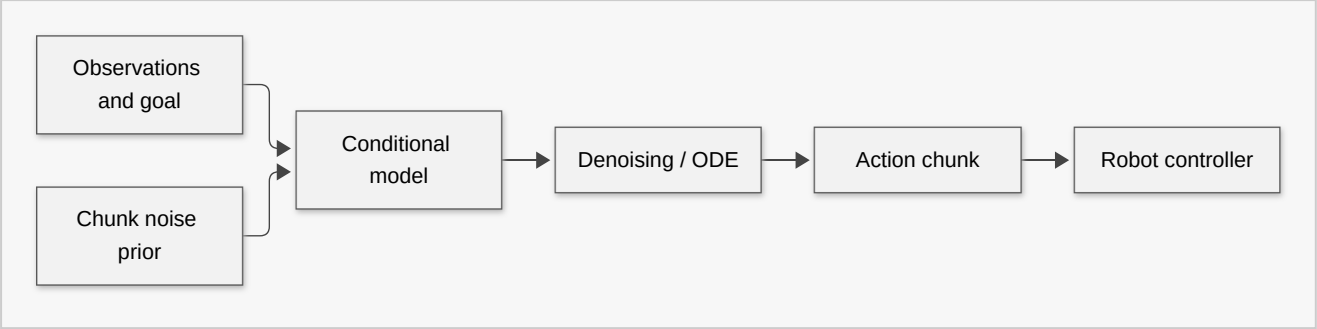
Action chunks reduce the need to replan every servo tick but may use stale observations during execution. Distinguish prediction horizon, execution horizon, and policy query rate. The chunk's command frame must agree with the actual controller interface.

4.1 ACT in One Picture

ACT uses a Transformer to predict a sequence of actions; the original formulation includes a latent-variable mechanism for demonstration variation. Chunks suit continuous structured demonstrations, but alignment and overlap policies still matter.

4.2 Diffusion Policy and Flow Matching

Diffusion Policy and Flow Matching can both represent multimodal action distributions, but **denoising dynamics or generative velocity fields are not robot joint velocities**.



For fair comparison, control observation modalities, demonstration budget, inference budget, controller rate, and evaluation scenarios. Offline action loss alone is insufficient.

5. Augmentation, Recalibration and Simulation

Geometric augmentation must transform images, object goals, reference trajectories, and action labels consistently. Rotating images without transforming the appropriate action target can create incorrect supervision. Simulated expert trajectories expand coverage, but contact, rendering, and latency gaps require separate validation.

5.1 Contact Events and Recovery Samples

Start from a uniform-sampling baseline before oversampling near first contact, grasp failure, or object drop. Event sampling changes the training distribution; document weights and always evaluate on an unbiased test distribution.

5.2 Safe Augmentation Boundaries

Do not propagate arbitrary initial conditions into physically inconsistent contact trajectories. Check reachability, collision, contact consistency, and actuator limits after rotation, translation, scaling, or timing perturbations; short physics replay is useful before admitting data.

6. A Shared Panda Tabletop Experiment

Task: in Isaac Lab or MuJoCo, use a Panda arm to grasp a randomized cube; collect scripted or teleoperated demonstrations and train BC, ACT, and a generative policy.

Control variables: camera view, task count, command interface, control rate, object split, and episode count. Simple BC is a required baseline for generative methods.

Metrics: success, contact failure, trajectory drift, recovery, policy inference latency, servo latency, and demonstrations used. Report seeds and failure modes rather than claiming one family always wins.

6.1 Minimum Experiment Matrix

Comparison	Purpose
One-step BC	Verify labels and controller
Chunked BC	Isolate chunking benefit
ACT	Test Transformer chunk prediction
Flow/Diffusion	Test multimodal action modelling
Recovery demos	Test closed-loop robustness

6.2 Debugging Checklist

Check timestamps, command/feedback confusion, camera extrinsics, action scale, latency, execution horizon, data splits, and initial-state coverage before scaling the model. Training-set action MSE is not a substitute for robot task success.

7. Unified Data Model: From Demonstrations to Action Chunks

7.1 Distinguish Latent State, Observations, and Commands

In real manipulation the full world state s_t (object motion, friction, contact modes) is usually not observed. We record images o_t^{cam}, joint feedback q_t, actual actuator commands a_t, and a goal g. A demonstration may be represented as

$$\tau_i = \{g_i, (o_t^{\text{cam}}, q_t, a_t, r_t^{\text{event}})_{t=0}^{T_i-1}\}.$$

The event label r_t^{event} may encode contact, slip, recovery, or completion; it need not be an RL reward. Ground-truth simulated object poses, future expert commands, or post-hoc outcome labels must not leak into the deployed policy observations. Keep issued commands distinct from delayed joint feedback.

Expert vs. behavior policy: Let π_E denote the expert and μ the actual data-collection policy. When demonstrations include exploratory actions, corrective trials or multiple operators, μ need not equal π_E. Preserve action provenance and evaluate on unbiased held-out episodes.

7.2 The Action-Chunk Prediction Problem

Construct conditioning information $c_t = (o_{t-L+1:t}, q_t, g)$ from an observation window, current proprioception and goal. The policy predicts an H-step sequence:

$$A_t = (a_t, a_{t+1}, \ldots, a_{t+H-1}), \qquad p_\theta(A_t \mid c_t).$$

Prediction horizon H is **not** the number of steps that must run open-loop. Execute only $h \le H$ steps, observe, then replan. Document control interval, inference call interval, queueing and interpolation. For example, 30-Hz vision and 100-Hz servo feedback do not imply that the large VLA must run at 100 Hz; it does require a safe interpolation, hold and timeout policy.

7.3 Time Alignment, Frames, and Action Normalization

At action time t_k^a, training inputs must already have been available. Accounting for acquisition/transport lag δ, choose a historical frame satisfying $t_j^{\text{image}} \le t_k^a - \delta$ rather than a future frame. For Cartesian pose commands, base-frame left-multiplied increments $T_{BE,t+1} = \Delta T_{B,t} T_{BE,t}$ differ from end-effector-frame right-multiplied increments $T_{BE,t+1} = T_{BE,t} \Delta T_{E,t}$.

Estimate action normalization statistics only on the training set:

$$\tilde{a} = (a - \mu_D)/(\sigma_D + \varepsilon), \qquad a_{\text{deployed}} = \tilde{a}(\sigma_D + \varepsilon) + \mu_D.$$

Different joints, gripper apertures, rotations and force commands require consistent units and independent safety limits. Never use test data to fit normalization parameters.

8. Behavior Cloning: Objectives, Intuition, and Failure Modes

8.1 Deterministic and Probabilistic BC

Given expert samples $(c, a) \sim d_E$, a deterministic baseline minimizes

$$\mathcal{L}_{\text{BC}}(\theta) = \mathbb{E}_{(c,a)\sim d_E} \|\pi_\theta(c) - a\|_2^2.$$

Under squared loss it converges toward the conditional mean $\mathbb{E}[a \mid c]$. This works for sufficiently unimodal behavior. If going left and going right around an obstacle are equally valid, the mean could collide with the obstacle. A probabilistic policy instead minimizes the negative log-likelihood

$$\mathcal{L}_{\text{NLL}}(\theta) = -\mathbb{E}_{(c,a)\sim d_E} \log p_\theta(a \mid c).$$

For a Gaussian distribution with fixed isotropic variance, NLL is MSE up to scaling and constants. Mixtures or generative action distributions can represent multiple modes. **High dimensionality alone is not proof that ordinary BC is inadequate.**

For general Gaussian $p_\theta(a \mid c) = \mathcal{N}(\mu_\theta(c), \Sigma_\theta(c))$, the NLL includes $\frac{1}{2}(a-\mu_\theta)^\top \Sigma_\theta^{-1}(a-\mu_\theta)$ and $\frac{1}{2}\log|\Sigma_\theta|$; it reduces to a weighted MSE only when the covariance is fixed.

8.2 Covariate Shift and Compounding Error

Offline BC learns on the expert occupancy d_{π_E}, while the deployed policy visits d_{π_θ}:

$$\min_\theta \mathbb{E}_{s\sim d_{\pi_E}} \ell(\pi_\theta(s),\pi_E(s)) \quad \not\Rightarrow \quad \min_\theta \mathbb{E}_{s\sim d_{\pi_\theta}} \ell(\pi_\theta(s),\pi_E(s)).$$

A small contact error may move the object to a state absent from expert training data. Collect intervention and recovery data; evaluate under deliberate safe perturbations, not solely on shuffled held-out action labels.

8.3 Chunk Loss and Smoothness Trade-offs

For variable-length episodes use a validity mask $m_{t,k}$ and optional horizon weight w_k:

$$\mathcal{L}_{\text{chunk}} = \frac{\sum_{t,k} m_{t,k} w_k \|\hat{a}_{t+k|t} - a_{t+k}\|_1}{\sum_{t,k} m_{t,k} w_k}.$$

A smoothness penalty $\lambda_s \sum_k \|\hat{a}_{k+1} - \hat{a}_k\|^2$ may suppress unwanted oscillation, yet can also erase necessary changes at contact onset or release. Measure task success and contact-force peaks before choosing it based on visual smoothness.

9. Interactive Imitation and Recovering from Failure

9.1 DAgger: Expert Queries on Learner-Visited States

DAgger gathers the states the current learner actually visits and requests safe expert labels. At iteration i, a mixture $\pi_i = \beta_i \pi_E + (1-\beta_i)\pi_{\theta_i}$ visits $s \sim d_{\pi_i}$; the expert supplies $\pi_E(s)$:

$$D_{i+1} = D_i \cup \{(s,\pi_E(s)) : s \sim d_{\pi_i}\}, \qquad \theta_{i+1} = \arg\min_\theta \sum_{(s,a)\in D_{i+1}} \ell(\pi_\theta(s),a).$$

The expert mixing probability β_i is often decreased gradually. Hardware requires human intervention authority, velocity/force limits and emergency stops. Merely retraining from static offline trajectories is not DAgger.

One optional event-biased sampler uses $w_t = 1 + \lambda \sum_j \exp[-(t-t_j)^2/(2\sigma^2)]$ around contact/slip events t_j.

9.2 Failures, Recoveries, and Event-Biased Sampling

Classify demonstrations as successful, recoverable failure, irrecoverable failure, or intervention. Annotate first contact, slip, drop, reacquisition and regrasp. Imitate only **safe and meaningful recovery actions**; unsafe failures may still be kept as negative outcome labels.

Compare uniform time sampling against event-biased sampling. A weighted loss is $\mathcal{L} = \sum_t w_t \ell_t / \sum_t w_t$, but modified sampling changes the training distribution. Always report unbiased test performance as well.

10. ACT, Diffusion Policy, Flow Matching, and RTC: Training vs. Execution

10.1 ACT: Conditional VAE and Transformer Chunks

Action Chunking with Transformers (ACT) predicts a sequence of commands with a Transformer. Its original formulation uses a conditional variational autoencoder (CVAE) to represent variation in demonstrations. An encoder approximates $q_\phi(z \mid A, c)$, and a decoder models $p_\theta(A \mid z, c)$:

$$\mathcal{L}_{\mathrm{ACT}} = \mathbb{E}_{z\sim q_\phi}[-\log p_\theta(A \mid z, c)] + \beta D_{\mathrm{KL}}(q_\phi(z \mid A, c)\|p(z)).$$

The KL term regularizes the posterior against the latent prior. At inference there is no future ground-truth action chunk A; use the declared prior or a deterministic representative. Large β can cause posterior collapse; tiny β can create a mismatch between training posterior and inference prior.

10.2 Diffusion Policy: Predict Noise in Action Space

Let A_0 be a demonstrated action chunk, $\epsilon \sim \mathcal{N}(0, I)$, and k the diffusion step. Construct

$$A_k = \sqrt{\bar{\alpha}_k}A_0 + \sqrt{1-\bar{\alpha}_k}\epsilon$$

and fit the conditional noise prediction objective

$$\mathcal{L}_{\mathrm{diff}} = \mathbb{E}_{A_0,c,k,\epsilon}\|\epsilon - \epsilon_\theta(A_k, k, c)\|_2^2.$$

At deployment, iterative denoising produces an action chunk. Execute a subset and replan from the next observation. The denoising step count, visual encoder and network architecture all affect latency; "generative" does not imply real-time.

10.3 Flow Matching: Generative Velocity Is Not Joint Velocity

Use data endpoint $A_1 \sim p_{\mathrm{data}}(\cdot \mid c)$ and noise $A_0 \sim \mathcal{N}(0, I)$. For a straight interpolation $A_\tau = (1-\tau)A_0 + \tau A_1$, one possible training loss is

$$\mathcal{L}_{\mathrm{FM}} = \mathbb{E}_{c,A_0,A_1,\tau}\|v_\theta(A_\tau, \tau, c) - (A_1 - A_0)\|_2^2.$$

Sample by solving $dA_\tau/d\tau = v_\theta(A_\tau, \tau, c)$. This vector field is a derivative in **generative time**, not the robot's physical $\dot{q}$. Path choices and time orientations vary by implementation. The continuity equation, conditional-expectation argument and OpenPI time convention are derived exclusively in the Flow Matching tutorial; this chapter focuses on the imitation-learning interface.

10.4 Fair Comparisons Against BC

Use the same demonstrations, observation encoder, action representation, prediction/execution horizons and evaluation protocol for one-step BC, chunked BC, ACT, Diffusion Policy and Flow Matching. Record data volume, model calls, generation steps, inference latency and closed-loop success. Do not mistake the benefit of action chunking for the benefit of a Transformer, or of extra test-time computation for the learning algorithm itself.

10.5 Why RTC Matters: From Synchronous Chunks to Asynchronous Control

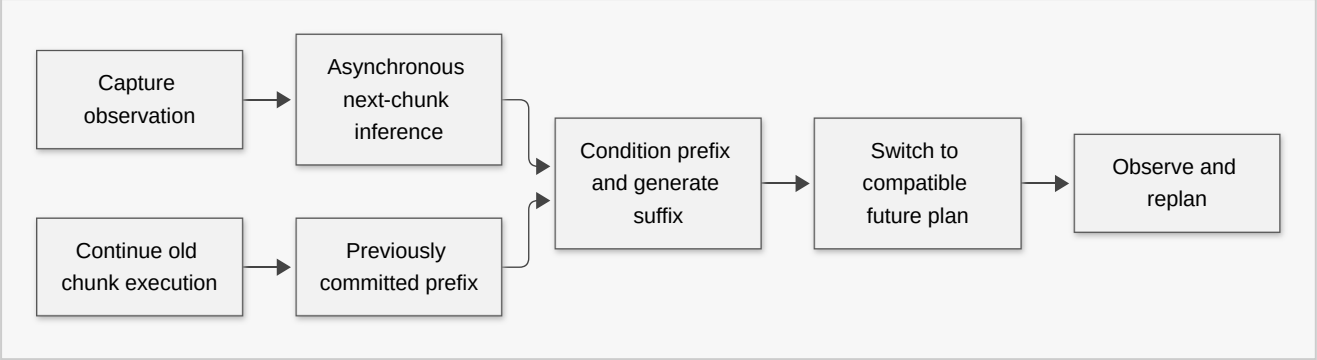
Real-Time Chunking (RTC) is not a new BC objective or a standalone motor policy. It is an execution/conditional generation technique for keeping diffusion- or flow-based chunk policies responsive despite inference latency. Synchronous chunk execution pauses at chunk boundaries.

Naive asynchronous switching removes the pause but may jump between different valid yet incompatible future trajectories. Temporal ensembling averages predictions, which can produce an invalid average of two legitimate modes in contact-rich tasks.

Let H be the prediction horizon, $s \le H$ the execution horizon, Δt the servo timestep, and δ the end-to-end latency from observation to available chunk. Measure inference delay in servo steps:

$$d \approx \left\lceil \frac{\delta}{\Delta t} \right\rceil, \qquad A_t = (a_t, \dots, a_{t+H-1}).$$

The ceiling is a **conservative engineering approximation**; the original RTC derivation uses $\lfloor \delta/\Delta t \rfloor$ and estimates future delay from a rolling history. For the asynchronous schedule in the original paper, require $d \le s \le H - d$: $d \le s$ permits inference to complete before the next handoff, and $d \le H - s$ leaves sufficient actions in the previous chunk to cover the inference delay. Leave additional margin for jitter, initialization, and timeouts. At 50 Hz ($\Delta t = 20$ ms), a measured 140 ms latency is roughly seven steps. Choosing $H = 50$ and $s = 20$ can leave enough old-chunk actions to cover inference, but does not replace timeout, packet-loss handling or emergency-stop logic.



10.6 Inference-Time RTC: Inpainting With a Frozen Prefix and Soft Overlap

Let Y be the old planned actions **aligned with the new chunk by absolute controller time**, and A^τ the new generated chunk at generative time $\tau \in [0, 1]$. The first d new-chunk actions correspond to time already committed to the old plan, so they must not change. Beyond that, some actions overlap the old plan but may be updated using new observations. The original RTC soft mask uses

$$W_i = \begin{cases} 1, & 0 \le i < d, \\ c_i(e^{c_i} - 1)/(e - 1), & d \le i < H - s, \\ 0, & H - s \le i < H, \end{cases}$$

$$c_i = \frac{H - s - i}{H - s - d + 1}.$$

The committed prefix receives weight one; the middle overlap decays; the last s actions are newly planned. **These are guidance weights inside generation, not post-hoc averaging of two action sequences.**

For a noise-to-data flow policy, a local fully-denoised endpoint prediction is

$$\widehat{A}^1 = A^\tau + (1 - \tau) v_\theta(A^\tau, \tau, c).$$

The original paper's pseudoinverse guidance (Eq. 2–4) can be written equivalently as a negative gradient of a weighted endpoint error:

$$\tilde{v}_\theta = v_\theta - \lambda_\tau \nabla_{A^\tau} \left[\frac{1}{2} \| W^{1/2} \odot (\widehat{A}^1 - Y) \|_2^2 \right].$$

The exact clipped guidance coefficient in the original paper is

$$r_\tau^2 = \frac{(1-\tau)^2}{\tau^2 + (1-\tau)^2}, \qquad \lambda_\tau = \min\left(\beta, \frac{1-\tau}{\tau\, r_\tau^2}\right).$$

The clipping parameter β prevents an overly large guidance correction. Implementations must protect numerical limits near $\tau = 0$ or $\tau = 1$. Integrate $A^{\tau+\Delta\tau} = A^\tau + \Delta\tau\, \tilde{v}_\theta$. This keeps the generated trajectory compatible with committed actions while still allowing future corrections. The correction involves a vector–Jacobian product during generative integration: **inference-time RTC does not require retraining, but adds inference computation.** The original method assumes a guidable continuous diffusion/flow sampler; an arbitrary deterministic ACT output cannot simply be passed through the same inpainting solver.

10.7 Training-Time RTC: Learn the Prefix Constraint From Demonstrations

Training-Time Action Conditioning for Efficient Real-Time Chunking incorporates the committed prefix into the imitation-learning objective. It learns

$$p_\theta(A_{d:H} \mid c, A_{0:d}),$$

where $A_{0:d}$ comprises indices $0, \ldots, d-1$ and the postfix $A_{d:H}$ comprises indices $d, \ldots, H-1$ (half-open interval notation). Given an expert chunk A, sample deployment-like delay $0 \le d < H$, flow time τ, and Gaussian noise ϵ. Use per-action flow times and inputs

$$\bar{\tau}_i = \begin{cases} 1, & i < d, \\ \tau, & i \ge d, \end{cases} \qquad \tilde{A}_i = \bar{\tau}_i A_i + (1 - \bar{\tau}_i)\epsilon_i.$$

The first d tokens are **clean ground-truth prefix actions**, while the remaining tokens learn to denoise. A compatible action transformer must support token-wise flow-time conditioning. Mask the loss so that only the postfix is supervised:

$$\mathcal{L}_{\mathrm{RTC-train}} = \mathbb{E}_{A,c,d,\tau,\epsilon} \left[\frac{1}{H-d} \sum_{i=d}^{H-1} \| v_\theta(\tilde{A}, \bar{\boldsymbol{\tau}}, c)_i - (A_i - \epsilon_i) \|_2^2 \right].$$

At deployment, supply the previously committed actions as a hard prefix and generate **only the editable suffix**. This avoids inference-time gradient guidance, but requires training/fine-tuning for a distribution of delays and does not support the original method's flexible soft overlap in exactly the same way. Training prefixes come from expert demonstrations; deployed prefixes come from previous model predictions, so rollout evaluation must cover this distribution shift.

This method belongs naturally in imitation learning because it changes **sample construction, token conditioning, and loss masking**. The full flow-matching mathematical derivation remains in Flow Matching. Sample d from measured latency statistics rather than assuming an arbitrary universal delay range; the cited $\pi_{0.6}$ experiments evaluate randomly sampled prefix lengths at both training and inference time.

10.8 Comparing RTC Against Temporal Ensembling and Naive Async

Execution mode	Waits at chunk boundaries?	How inter-chunk consistency is handled	Changes training?
Synchronous chunking	Usually yes	Replan after a pause	No
Naive asynchronous	Not necessarily	Immediate switch, possible jerk/mode jump	No
Temporal ensembling	Can be asynchronous	Average overlapping predictions; can fail with multimodality	No
Inference-time RTC	No	Frozen committed prefix + soft generative guidance	**No**, but extra inference cost
Training-time RTC	No	Prefix-conditioned denoising; hard prefix at inference	**Yes**, cheaper inference

Minimum evaluation: Fix the same flow policy, task seeds, controller rate, and safety stack. Compare all five execution modes with 0/100/200 ms injected latency. Report task success, completion time, task throughput, inference p50/p95 latency, missed servo deadlines, boundary jump $\|a_t - a_{t-1}\|$, acceleration/jerk, contact failure, and drop recovery. Distinguish model compute time, handoff delay, and physical execution time. Offline FPS is not enough.

References: RTC original paper (NeurIPS 2025), training-time RTC (2025), Physical Intelligence project/videos, and LeRobot RTC documentation. Extensions such as Soft RTC (2026) and DiscreteRTC (2026) explore soft overlap priors and native inpainting for discrete action samplers; their reported benefits are not universal guarantees for the original RTC method.

11. Human Video Retargeting, Contact, and Augmentation

11.1 Human-to-Robot Is Not Pixel Imitation

A realistic pipeline is hand/tool pose recovery → scale and frame calibration → embodiment-specific retargeting → contact event detection → short physics replay and trajectory repair → policy training. Humans and robot hands differ in kinematics, reachability and contact friction. One task-dependent objective is

$$\mathcal{L}_{\text{retarget}} = w_p \|p_{\text{robot}} - p_{\text{target}}\|^2 + w_R d_{SO(3)}(R_{\text{robot}}, R_{\text{target}})^2 + w_c \mathcal{L}_{\text{contact}} + w_j \mathcal{L}_{\text{limits}}.$$

Define $\mathcal{L}_{\text{contact}}$ for the task, e.g. contact point error, surface normals, grasp maintenance or object slip. Mere proximity does not certify physical contact.

11.2 Augmentation Must Preserve Geometry and Physics

When perturbing object position, scale or orientation, transform task goals, observable geometry, trajectories and reference-frame-dependent labels consistently. Rotating an image without adjusting Cartesian action labels is typically invalid. Contact phases should be physically replayed to check penetration, limits, slip and unreasonable forces before accepting augmented episodes.

12. Training and Evaluation Protocol

12.1 Offline Metrics, Closed-Loop Success, and Recovery

Test	Report	Does not replace
Held-out demonstrations	NLL / action error and chunk alignment	Closed-loop task success
Randomized starts	Success, completion time, collision/contact failures	Perturbation tests
Drop or disturbance	Recovery success, recovery latency, regrasp count	Nominal performance
Held-out objects/scenes	Generalization success with grouped splits	Data leakage checks
Runtime	Policy p50/p95 latency, actuation latency	A bare GPU FPS number

For k successes in N independent episodes, report $\hat{p} = k/N$ and a confidence interval. The rough standard error is $\sqrt{\hat{p}(1-\hat{p})/N}$; use a Wilson interval for small N or near-boundary success rates. Frames from the same episode are not independent trials.

12.2 A Shared Panda Tabletop Benchmark

Use the same Panda arm, cube, and tabletop in simulation. First validate control labels, then compare one-step versus chunked BC, then ACT versus diffusion/flow, and finally add recovery demonstrations and contact perturbations. When transferring across Isaac and MuJoCo, disclose differences in contact solvers, actuator dynamics and timing.

Chapter ownership: this tutorial owns demonstration quality, supervised imitation, distribution shift and action-policy comparisons. Complete PPO/SAC derivations belong to their respective chapters; formal FM derivations belong to the Flow Matching tutorial; the visual model toolbox belongs to Visual Perception and State Estimation.

13. Where This Fits and References

This chapter bridges camera calibration, visual state estimation, Flow Matching, and GR00T / SONIC.

- ACT / ALOHA
- Diffusion Policy
- DAgger
- Flow Matching for Generative Modeling
- LeRobot

Verification boundary: this chapter is a methods and experiment-design primer, not a claim that those policies were reproduced in the shared Panda setup.